# Panel de Canales · Plan por fases para Claude Code

Objetivo: plataforma multicanal (red de canales) donde el equipo vea calendario, estado de cada video, estadísticas y archivos, y que automatice el flujo de producción manteniendo aprobación humana.

## Contexto (leer antes de empezar)

- **Multicanal desde el día 1:** el proyecto no lleva el nombre de ningún canal. Dinero Oculto es el primer canal (canal #1) de una red que va a escalar. Nada de marca, colores, horarios o reglas de un canal puede quedar fijo en el código: todo vive en la configuración de cada canal.
- Canal: Dinero Oculto (finanzas sencillas + cómo funcionan empresas/negocios + fortunas). 3 videos/semana (lun/mié/vie, 12:00 America/Bogota), 1 largo + 4 TikToks (1080x1920) + 3 Shorts por video.
- Fuentes en el proyecto: `Mi-Canal/yt/` (hookscore_es.py, title_es.py, chapters_es.py, deadair.py, retention.py, swipe_es.py, hooks_es.json, voice.md, ESTRATEGIA_DINERO_OCULTO.md, AGENDA_PUBLICACION_10_VIDEOS.md) y `Mi-Canal/remotion/` (Canal.tsx, Tiktok.tsx, theme.ts, GUIA_ESTILO*.md).
- Marca del panel: neutra (negro + acento configurable). Cada canal define su propia marca (Dinero Oculto: negro + dorado #E9B949), idioma, zona horaria, horario de publicación y pilares. Idioma de la UI: español.

## Stack

Next.js (App Router, TypeScript) · Tailwind · MySQL de Hostinger (Prisma) · Auth.js con enlace por correo · todo desplegado en **Hostinger** · tareas en el mismo hosting (sin VPS) · Remotion fuera del panel.

## Infraestructura Hostinger (cuenta existente)

- **Panel web:** hosting Node.js (usuario u945139586). Usar un dominio provisional de Hostinger (tipo *.hostingersite.com, sin subdominio de 9sm.site) y cambiarlo por el dominio definitivo más adelante; desplegar con build de Node.js y variables de entorno en el hosting (nunca en el repo).
- **Base de datos:** MySQL del mismo hosting (host 127.0.0.1:3306 desde la app). Sin Postgres ni RLS: los permisos por rol se aplican en el servidor (middleware y capa de acceso a datos).

- **Sin VPS:** todo corre en el hosting Node.js. Las tareas periódicas usan cron del hosting o rutas internas del panel; la lógica de hookscore/title se porta a TypeScript (o se ejecuta con Python si el hosting lo permite).
- **Render de video (Remotion), voz e imágenes:** fuera del panel. Se hacen en la PC o con las herramientas de IA; el panel solo registra el estado y los enlaces de salida.
- **Archivos grandes:** los videos viven en Drive; el panel guarda enlaces, no los archivos.

## Reglas generales

1. Nada se publica sin aprobación humana (estado `aprobado` firmado por un usuario con rol).
2. Cifras sobre personas/empresas reales: tarea de verificación con fuente obligatoria antes de `listo_para_producir`.
3. Ninguna clave en el repo; variables de entorno. Límite de gasto por video para créditos de IA.
4. Todo código y UI parametrizado por canal (selector de canal arriba, vista "Red" con todos los canales). Añadir un canal nuevo = crear registro + importar su agenda, sin tocar código.
5. Cada fase termina con: tests básicos, README actualizado, despliegue en Hostinger, y demo con criterios de aceptación cumplidos.

## Modelo de datos (base)

`canales(id, slug, nombre, idioma, zona_horaria, hora_publicacion, color_acento, pilares, reglas)` · `membresias(user_id, canal_id, rol)` · `users(id, email, nombre, es_superadmin)` · `videos(canal_id, id, titulo, pilar, fecha_publicacion, estado, gancho, score_gancho, fuentes_ok)` · `tareas(id, video_id, tipo, responsable, estado, vence)` · `activos(id, video_id, tipo, url, version)` · `publicaciones(id, video_id, plataforma, url, programada, publicada)` · `metricas(id, publicacion_id, fecha, vistas, retencion, ctr, likes, comentarios)` · `ejecuciones(id, tipo, estado, log, costo)`. Estados de video: idea → guion → verificación → producción → revisión → aprobado → programado → publicado. Roles por canal (tabla membresias): admin, productor, revisor, lector. Superadmin ve toda la red. Toda tabla con datos de contenido lleva `canal_id` y toda consulta se filtra por los canales del usuario.

## Fase 0 · Fundamentos

- Repo, Next.js + Tailwind, tema negro/dorado, layout con navegación.
- MySQL (Hostinger): esquema anterior con Prisma y migraciones; permisos por rol en el servidor.
- Auth.js con enlace por correo (SMTP de Hostinger).
- Despliegue en Hostinger Node.js en el dominio provisional con variables de entorno.

- Cargar el calendario inicial desde AGENDA_PUBLICACION_10_VIDEOS.md (seed). Aceptación: login funcional, 16 videos cargados, roles probados.

## Fase 1 · MVP del panel

- Vista Calendario (mes/semana) con color por tipo: publicación, producción (3 días antes), reunión.
- Tablero Kanban por estado con arrastrar y soltar y checklist por video.
- Ficha de cada video: título, gancho, fuentes, archivos, enlaces (Drive, YouTube, TikTok).
- Gestión de usuarios y roles; registro de actividad. Aceptación: el equipo mueve un video entre estados y el cambio se ve en tiempo real; permisos por rol verificados.

## Fase 2 · Datos y estadísticas

- Sincronizar Google Calendar (lectura/escritura de eventos del canal).
- Estadísticas de YouTube (vistas, retención, CTR, suscriptores) por video con gráficos.
- Estadísticas de TikTok cargadas por CSV/formulario (API después).
- Tablero de resultados: mejor hora, mejor pilar, comparación de ganchos. Aceptación: panel con métricas reales de al menos 1 video publicado y exportable a CSV.

## Fase 3 · Pipeline de producción asistida

- Tarea del panel (TypeScript) que ejecuta hookscore/title lint y guarda el puntaje en el video.
- Generador de borrador de guion con reglas de voice.md y checklist de verificación de cifras.
- Cola de trabajos: voz (ElevenLabs), imágenes (Higgsfield, con tope de créditos), subtítulos.
- Render Remotion (largo + 4 TikToks zona segura + 3 Shorts) fuera del panel; el panel registra estado y enlaces de Drive. Aceptación: desde un video en estado `producción`, un botón lanza el pipeline y deja los enlaces de salida adjuntos con log y costo.

## Fase 4 · Publicación

- Subida a Drive (carpeta del video) y generación automática de ficha (YouTube y TikTok sin enlaces en comentarios).
- Programación en YouTube vía API; TikTok manual guiado (checklist y texto listo) hasta tener API aprobada.
- Aprobación obligatoria antes de programar; notificación al equipo. Aceptación: un video aprobado queda programado en YouTube y con ficha en Drive.

## Fase 5 · Automatizaciones y reportes

- Investigación semanal de referencias (vidIQ + web) guardada como informe en el panel.
- Alertas: video atrasado, falta de colchón (<2 videos listos), cifras sin verificar.
- Resumen semanal automático al equipo (aprobado por un admin). Aceptación: informe semanal y alertas funcionando una semana completa.

## Fase 6 · Endurecimiento

- Auditoría de permisos, logs, copias de seguridad, control de costos, manejo de errores y reintentos.
- Documentación de operación y guía para el equipo.

## Orden sugerido y estimación

Fase 0–1: 2–4 días · Fase 2: 2–3 días · Fase 3: 4–7 días · Fase 4: 3–5 días · Fases 5–6: 3–4 días. Entregar y probar con el equipo al final de cada fase antes de seguir.